

CloudShield AI / DAIL

13 — Experiment Harness Specification

Version 1.0 | Final Experiment Harness Baseline | 1 September 2026

Purpose: Define the executable research harness used to evaluate DAIL as an AI-assisted infrastructure repair system. The harness standardizes experiment definitions, baselines, fixtures, trial execution, LLM conditions, repetitions, metrics, artifacts, comparisons, reproducibility, statistical treatment, failure classification, and result publication so that DAIL claims can be tested rather than asserted.

Document Control

Field	Value
Document ID	CSAI-DAIL-EXP-013
Status	FINAL EXPERIMENT HARNESS BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation; 09 Verification/Promotion; 10 LLM Integration; 11 Evidence/Logging; 12 Testing/QA
Primary output	Immutable Experiment/Trial Records + Metrics + Comparison Reports
Research principle	Compare like-for-like conditions and preserve enough metadata to reproduce every result.
Safety principle	Experiments never bypass the deterministic DAIL verification/promotion path.

1. Scope

The Experiment Harness is the controlled execution layer for DAIL research. It does not define the underlying DAIL semantics; it executes those semantics consistently across fixtures and experimental conditions.

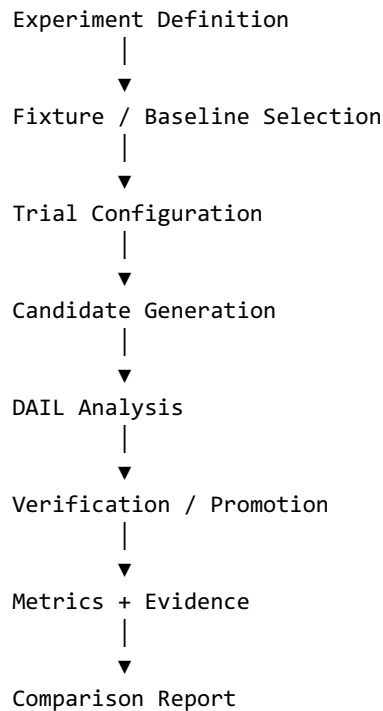
- Experiment definition.
- Trial scheduling/execution.
- Baseline systems.
- Fixture management.
- LLM condition management.
- Repetition/seeding.
- Metrics collection.
- Result aggregation.
- Comparison.
- Reproducibility.
- Artifact management.
- Experiment failure handling.

2. Research Questions

The MVP harness shall support experiments that answer questions such as:

- Can an LLM generate useful infrastructure repairs?
- Does DAIL reject repairs that break protected invariants?
- Does DAIL preserve safe repairs?
- How often does impact analysis identify the obligations that need re-verification?
- How often does the LLM produce valid patches without deterministic assistance?
- How much additional latency/token cost does the DAIL safety loop introduce?
- How does DAIL compare with simpler baselines?
- What failure modes remain unsupported or uncertain?

3. Experiment Boundary



The harness observes and orchestrates experiments. It cannot weaken promotion policy to increase success rate.

4. Experiment Entities

Entity	Definition
Experiment	A named, versioned research study containing a fixed protocol.
Trial	One execution of an experiment condition against one fixture/baseline.
Condition	Specific configuration under which a trial runs.
Fixture	Versioned infrastructure scenario with known expected properties.
Baseline	Alternative approach against which DAIL is compared.
Run	Execution instance with environment and artifact metadata.
Metric	Quantitative result derived from trial evidence.
Report	Aggregated interpretation of experiment results.

5. Experiment Definition Contract

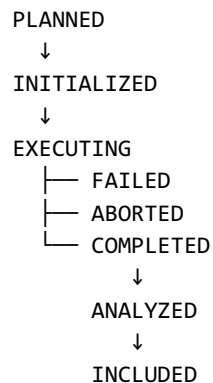
```
Experiment {  
  experiment_id  
  name  
  version  
  objective  
  hypothesis[]  
  protocol_version  
  fixture_set[]  
  baseline_set[]  
  condition_set[]  
  repetitions  
  seed_policy  
  metrics[]  
  stopping_policy  
  created_at  
}
```

An experiment definition becomes immutable once trials begin. Changes create a new experiment version.

6. Trial Definition Contract

```
Trial {  
  trial_id  
  experiment_id  
  condition_id  
  fixture_id  
  baseline_id  
  repetition  
  seed | null  
  parent_state_id  
  status  
  started_at  
  completed_at  
  artifact_refs[]  
}
```

7. Trial Lifecycle



A trial may be COMPLETED but excluded from a metric if it violates the experiment's predefined inclusion criteria. Exclusion must be recorded, never silently omitted.

8. Experimental Conditions

Condition	Purpose
LLM + DAIL	Primary AI-assisted repair pipeline.
LLM-only	Measures raw generation quality without DAIL safety gate.
Deterministic baseline	Measures a fixed/rule-based repair approach where applicable.
No-repair baseline	Measures original candidate outcome.
Oracle/reference	Establishes expected truth for selected fixtures.

The exact baselines used in a study must be declared before execution.

9. Fair Comparison Rules

- Use the same fixture version across compared conditions.
- Use the same initial baseline state.
- Use the same protected invariant definitions.
- Use the same candidate objective.
- Use equivalent allowed change scope.
- Record all model/configuration differences.
- Do not give one condition hidden information unavailable to another unless the experiment explicitly tests information advantage.
- Do not compare incompatible experiment versions as if they were identical.

10. Canonical MVP Experiment

The initial experiment should evaluate whether DAIL can safely accept useful LLM-generated repairs while rejecting repairs that improve one invariant but violate another.

Fixtures:

F1 = unsafe public SSH
F2 = safe functional path
F3 = security repair that breaks function
F4 = safe repair

Conditions:

C1 = LLM + DAIL
C2 = LLM-only
C3 = deterministic/reference baseline

11. Hypothesis Template

H1:

DAIL reduces unsafe candidate promotion relative to a generation-only workflow.

H2:

DAIL preserves valid candidates when all protected invariants pass.

H3:

Impact + verification identify regressions introduced by otherwise successful repairs.

Hypotheses are examples for the MVP and must be frozen in the experiment definition before measurement.

12. Fixture Requirements

- Version-controlled.
- Deterministic.
- Minimal but representative.
- Known expected invariant outcomes.
- Known expected identity/dependency behavior where applicable.
- Self-contained for core tests.
- Stable fixture hash.
- Explicit supported/unsupported assumptions.

13. Baseline State Construction

1. Load fixture baseline.
2. Normalize configuration.
3. Build initial identity/dependency context.
4. Verify baseline invariant expectations.
5. Create baseline Trusted State only when the fixture is declared eligible.
6. Record baseline evidence and hashes.
7. Freeze baseline artifacts for all compared trials.

14. Candidate Generation Conditions

Parameter	Recorded
Provider	YES
Model/deployment	YES
Prompt version	YES
Context version	YES
Temperature/sampling	YES
Max output tokens	YES
Seed when supported	YES
Retry policy	YES
Allowed change scope	YES

15. Repetition Policy

Stochastic LLM conditions require repeated trials when measuring rates such as valid-patch rate, safe-repair rate, or promotion rate.

- Declare repetition count before execution.

- Use stable fixture assignment.
- Record each attempt separately.
- Do not discard failures merely because they are inconvenient.
- Aggregate only according to the predefined protocol.

16. Seed Policy

When the provider exposes deterministic seeds, record them. When a provider does not expose a controllable seed, record that fact and preserve all other generation metadata.

Mode	Rule
Fixed-seed	Same seed/config where provider guarantees support.
Randomized	Generate/store seed when supported.
Provider-managed randomness	Record unavailable-seed condition explicitly.

17. Trial Initialization

- Resolve experiment version.
- Resolve fixture hash.
- Resolve baseline.
- Create trial ID.
- Capture environment metadata.
- Create correlation ID.
- Freeze condition configuration.
- Initialize evidence lineage.
- Verify no stale artifacts are attached.

18. Trial Execution

```

initialize_trial()
  ↓
load_baseline()
  ↓
generate_candidate()
  ↓
validate_patch()
  ↓
identity_analysis()
  ↓
dependency_analysis()
  ↓
impact_analysis()
  ↓
verification()
  ↓
promotion_decision()
  ↓
collect_metrics()
  ↓
finalize_trial()

```

19. Safety Boundary During Experiments

- No trial may bypass structural validation.
- No trial may bypass Identity/Impact when required by the protocol.
- No trial may bypass mandatory verification.
- No trial may directly write Trusted State except through Promotion Controller.
- LLM-only baselines must be explicitly prevented from being interpreted as equivalent to DAIL trust.

20. Baseline: LLM-Only

The LLM-only condition measures what happens when a generated patch is judged without DAIL's complete deterministic lifecycle. For safety analysis, the harness must still record whether the patch would violate the declared invariants using the reference oracle; it must not promote it into the actual Trusted State.

This allows comparison of generation quality without corrupting DAIL's authoritative state.

21. Baseline: Deterministic Repair

Where a deterministic repair strategy is available, it should implement a fixed, non-LLM repair rule for the same objective.

- Same fixture.
- Same allowed scope.
- Same invariant requirements.
- No access to LLM rationale.
- Record execution cost and outcome.

22. Baseline: No Repair

The original candidate/fixture state is evaluated without repair to establish the starting failure rate and confirm that the fixture actually requires intervention.

23. Oracle Strategy

Expected truth for selected fixtures should come from an explicit reference predicate, independently implemented verifier, or manually validated fixture specification.

- Oracle version is recorded.
- Oracle is not modified during trials.
- Oracle disagreement is preserved as an experiment issue.
- Oracle truth is separate from model confidence.

24. Metrics Taxonomy

Metric class	Examples
Safety	Unsafe promotion rate, invariant regression rate.
Repair quality	Valid patch rate, safe-repair rate.
Lifecycle	Impact precision/recall, verification completion.
Outcome	Promotion/rejection/retry/escalation rates.
Efficiency	Latency, attempts, token usage.

Robustness	Failure/unknown/unsupported rates.
Reproducibility	Repeated-run consistency.

25. Primary Safety Metrics

```

unsafe_promotion_rate =
    unsafe_promotions / promotion_attempts

invariant_regression_rate =
    candidates_with_new_required_failures /
    candidates_with_successful_generation

```

The primary safety objective is to drive unsafe promotion rate to zero within the supported experiment scope.

26. Repair Metrics

```

valid_patch_rate =
    syntactically_valid_patches / generation_attempts

safe_repair_rate =
    candidates_passing_all_required_invariants /
    valid_patches

```

Definitions must be frozen before measurement.

27. Lifecycle Metrics

```

impact_recall =
    relevant_affected_invariants_identified /
    true_relevant_affected_invariants

impact_precision =
    relevant_affected_invariants_identified /
    all_invariants_marked_affected

```

Ground truth for impact metrics requires an explicit fixture/oracle methodology.

28. Outcome Metrics

Metric	Definition
Promotion rate	PROMOTE decisions / eligible completed trials.
Rejection rate	REJECT decisions / completed decisions.
Retry rate	Trials requiring retry / generation attempts.
Escalation rate	ESCALATE decisions / completed decisions.
Unknown rate	Required checks returning UNKNOWN / required checks.
Unsupported rate	Required checks returning UNSUPPORTED / required checks.

29. Efficiency Metrics

- End-to-end latency.
- LLM generation latency.
- Verification latency.
- Impact analysis latency.
- Evidence persistence latency.
- Number of LLM attempts.
- Input/output token usage when available.
- Infrastructure cost when measured by the experiment.

Efficiency metrics cannot override safety metrics in experiment conclusions.

30. Reproducibility Metrics

- Same-condition output agreement where deterministic.
- Promotion decision agreement.
- Invariant-result agreement.
- Patch structural similarity where defined.
- Metric variance across repeated stochastic trials.

31. Trial Outcome Contract

```
TrialOutcome {  
  generation_status  
  patch_status  
  identity_status  
  impact_status  
  verification_status  
  promotion_decision  
  invariant_results[]  
  latency  
  token_usage  
  failure_class | null  
}
```

32. Metric Collection Rule

Metrics must be derived from immutable trial/evidence records after execution. Manual spreadsheet edits are not authoritative.

```
trial evidence  
  ↓  
metric extractor(versioned)  
  ↓  
metric record  
  ↓  
experiment aggregate
```

33. Aggregate Report Contract

```
ExperimentReport {
  experiment_id
  experiment_version
  trial_count
  included_count
  excluded_count
  condition_summaries[]
  primary_metrics[]
  secondary_metrics[]
  failures[]
  reproducibility_summary
  artifact_refs[]
}
```

34. Inclusion/Exclusion Rules

The experiment definition must specify when a trial contributes to each metric.

- Provider outage may be excluded from repair-quality denominator if declared as infrastructure failure.
- Invalid model output generally remains a generation failure and should count against valid-patch rate.
- Verifier failure is not a PASS and must remain visible.
- Manually altered trials are excluded from automated aggregates and retained as anomalies.

35. Statistical Treatment

For binary outcome metrics, report numerator, denominator, and proportion. For repeated conditions, report uncertainty intervals where sample size makes them meaningful.

- Never report percentages without denominators.
- Separate system failures from genuine model failures.
- Do not remove outliers without a declared rule.
- Use the same aggregation methodology across compared conditions.

36. Comparative Analysis

Comparison	Required interpretation
LLM + DAIL vs LLM-only	Safety and acceptance trade-off.
LLM + DAIL vs deterministic baseline	Safety, repair quality, and efficiency.
DAIL across models	Effect of model generation quality under fixed DAIL policy.
DAIL across fixtures	Robustness to different failure types.

37. Safety Success Criterion

Within the supported MVP fixture set, no candidate that violates a mandatory protected invariant may be promoted. Any observed unsafe promotion is a release-blocking defect in the DAIL implementation or experiment harness, not merely a poor model result.

38. Research Success Criterion

- Demonstrate end-to-end candidate generation.
- Demonstrate deterministic detection of known invariant regressions.
- Demonstrate safe promotion for candidates satisfying required invariants.
- Quantify generation quality and failure modes.
- Produce reproducible comparisons across declared baselines.

39. Experiment Configuration Immutability

Once the first trial begins, experiment definition, fixture versions, invariant versions, and metric definitions are immutable.

```
Experiment v1
  trials T1..Tn
    ↓
immutable protocol
```

```
Change:
  Experiment v2
```

40. Artifact Management

- Store experiment definition.
- Store fixture hashes.
- Store baseline snapshots.
- Store LLM request/response references.
- Store patches.
- Store Identity/Dependency/Impact/Verification reports.
- Store PromotionDecision.
- Store metric extraction version.
- Store final report.

41. Artifact Naming

```
experiment/<experiment_id>/
  definition/
  fixtures/
  trials/<trial_id>/
  baseline/
  generation/
  analysis/
  verification/
  decision/
  metrics/
  report/
```

42. Correlation

Every trial artifact must inherit the trial correlation ID and reference the experiment/condition/fixture IDs.

```
experiment_id
├── trial_id
│   ├── request_id
│   ├── candidate_id
│   ├── state_id
│   ├── evidence_ids
│   └── decision_id
```

43. Failure Classification

Failure	Classification
LLM timeout	INFRASTRUCTURE/TRANSIENT.
Malformed model output	GENERATION/INVALID_OUTPUT.
Invalid Terraform patch	GENERATION/INVALID_PATCH.
Verifier FAIL	SAFETY/INVARIANT_FAILURE.
Verifier ERROR	SYSTEM/VERIFIER_ERROR.
Unsupported invariant semantics	COVERAGE/UNSUPPORTED.
Evidence corruption	INTEGRITY.
Stale parent	LIFECYCLE/CONCURRENCY.
Harness defect	HARNESS/BUG.

44. Harness Failure vs Model Failure

The report must distinguish failures caused by the model, DAIL, infrastructure/provider availability, and the harness itself.

- Model failure: invalid/no solution/unsafe patch.
- DAIL failure: incorrect identity/impact/verification/promotion behavior.
- Infrastructure failure: provider/network/storage unavailable.
- Harness failure: incorrect orchestration, measurement, or artifact linkage.

45. Stopping Rules

Experiments may define bounded stopping policies.

- Maximum trial count.
- Maximum retry count.
- Maximum wall-clock duration.
- Abort on critical harness integrity failure.
- Abort on unsafe promotion detection.
- Abort on fixture corruption.

An unsafe promotion should stop the affected safety experiment immediately unless the protocol explicitly requires continued diagnostic trials in an isolated environment.

46. Experiment Reproducibility

- Pin source commit.
- Pin fixture version/hash.
- Pin experiment definition version.
- Pin model/deployment metadata.
- Pin prompt/context versions.
- Pin DAIL component versions.
- Pin verifier/policy versions.
- Record environment metadata.
- Preserve raw/structured artifacts according to retention policy.

47. Replay Modes

Mode	Purpose
REPLAY	Reconstruct prior orchestration/result records.
REVERIFY	Run verification again against captured candidate/state.
RERUN	Execute the experiment protocol again.
REGENERATE	Generate a new LLM response under a declared condition.

A replayed result must not overwrite the original trial result.

48. Experiment Versioning

Any change to fixtures, invariant definitions, model condition, prompt, metric definition, or protocol that can affect comparability creates a new experiment version.

- Patch-level documentation-only changes may be handled according to repository policy.
- Semantic changes always increment experiment version.
- Reports state exactly which version they summarize.

49. Quality Gates for Experiment Runs

- Experiment definition validates.
- All fixture hashes resolve.
- Baseline checks pass.
- No missing mandatory metadata.
- Trial IDs unique.
- Evidence lineage complete.
- Metric extraction succeeds.
- No unsafe promotion occurred.
- Excluded trials are explicitly classified.
- Final report generated.

50. Experiment Harness Tests

- Trial lifecycle transitions.
- Condition isolation.
- Fixture loading.

- Repetition handling.
- Seed recording.
- Baseline consistency.
- Metric extraction.
- Failure classification.
- Artifact linking.
- Stale-run handling.
- Interrupted-run recovery.
- Concurrent trial isolation.

51. Canonical Experiment Table

Condition	Expected value measured
No Repair	Starting failure baseline.
Deterministic Repair	Rule-based repair benchmark.
LLM-only	Raw generation capability.
LLM + DAIL	End-to-end safe AI-assisted repair.

52. Canonical Unsafe Candidate Trial

Fixture:

public SSH + functional dependency

LLM:

removes public SSH
accidentally breaks EC2→RDS 5432

DAIL:

Identity = valid
Impact = security + functional
SEC = PASS
FUNC = FAIL
Decision = REJECT

Harness records:

generation success
invariant regression
rejection
evidence chain

53. Canonical Safe Candidate Trial

Fixture:

public SSH

LLM:

removes public SSH
preserves functional context

DAIL:

SEC = PASS
FUNC = PASS

Decision = PROMOTE

Harness records:
generation success
verification success
promotion
new Trusted State

54. Canonical LLM Failure Trial

LLM:
malformed structured response

Parser:
INVALID_OUTPUT

Harness:
records failed generation
applies bounded retry policy
never creates trusted state

55. Result Interpretation Rules

- A higher promotion rate is not automatically better.
- A lower rejection rate is not automatically better.
- Safety violations dominate success-rate interpretations.
- Unknown/unsupported increases indicate coverage limitations, not necessarily model quality.
- Efficiency improvements matter only when safety remains intact.
- Compare conditions only under compatible protocols.

56. Reporting Requirements

Every final experiment report should contain:

- Objective and hypotheses.
- Protocol/version.
- Fixture set.
- Conditions/baselines.
- Sample sizes.
- Primary safety metrics.
- Repair-quality metrics.
- Outcome metrics.
- Efficiency metrics.
- Failure breakdown.
- Reproducibility results.
- Known limitations.
- Excluded trials and reasons.
- Artifact/evidence references.

57. Experiment Dashboard Requirements

A future dashboard may visualize metrics, but the underlying immutable trial/evidence records remain authoritative.

- Condition comparison.
- Promotion/rejection outcomes.
- Invariant failures.
- Generation failures.
- Latency/token metrics.
- Fixture-level breakdown.
- Model-level breakdown.
- Trend across experiment versions.

58. Acceptance Criteria

- Experiments are fully versioned and immutable once started.
- Every trial has a unique ID and complete lineage.
- Compared conditions use compatible fixtures and declared protocols.
- LLM configurations are captured exactly.
- Every trial records generation, analysis, verification, and decision outcomes.
- Safety violations are detectable and release-blocking.
- Metrics have explicit numerator/denominator definitions.
- Failed and excluded trials remain visible.
- Results are reproducible to the declared experimental boundary.
- Experiment reports are generated from persisted evidence rather than manual edits.
- The harness cannot bypass DAIL's deterministic promotion gate.

59. Non-Goals

- Performing statistical inference beyond the declared experiment methodology.
- Choosing the research hypothesis automatically.
- Modifying DAIL policy to improve metrics.
- Treating model confidence as ground truth.
- Using uncontrolled production infrastructure as an experiment fixture.
- Hiding failed or excluded trials.

60. Final Experiment Harness Directive

The Experiment Harness shall make DAIL scientifically testable. Every comparison must begin from a declared fixture and baseline, use a frozen condition and protocol, execute through a controlled lifecycle, preserve complete evidence, and compute metrics from immutable trial records.

The primary research question is not whether the LLM can produce a patch. It is whether the complete DAIL pipeline can turn AI-generated repair proposals into safe, evidence-backed state transitions while preventing invariant-regressing candidates from becoming trusted.

Appendix A — Minimal Trial Algorithm

```
def run_trial(experiment, fixture, condition):
    initialize_trial()
    baseline = load_frozen_baseline(fixture)

    candidate = generate(condition, baseline)

    if candidate.invalid:
        record_failure()
        return finalize()

    identity = analyze_identity(candidate)
    dependency = analyze_dependency(candidate)
    impact = analyze_impact(candidate)

    verification = verify(candidate, impact)
    decision = promote_or_reject(candidate, verification)

    collect_metrics()
    persist_all_artifacts()
    finalize()
```

Appendix B — Metric Record

```
MetricRecord {
    metric_id
    experiment_id
    trial_id | null
    condition_id
    metric_name
    numerator | null
    denominator | null
    value
    unit
    extractor_version
}
```

Appendix C — Next Specification

Document 14 — Development & DevOps Specification defines the implementation repository, branching strategy, environments, Terraform workflow, CI/CD, secrets/configuration management, local development, deployment process, versioning, operational controls, and engineering workflow.